



Instrucciones SQL

Proyecto de Producción

Bases de Datos Relaciones

¿Qué es una base de datos relacional?

¿Cuales son las no relacionales?

Las Bases de Datos SQL pueden ser de de tipo relacionales o no relacionales. En el caso de las relacionales se estructura la información en tablas, filas y columnas. Las mismas tienen la capacidad de establecer relaciones de la información entre distintas tablas por medio de **claves foráneas**.

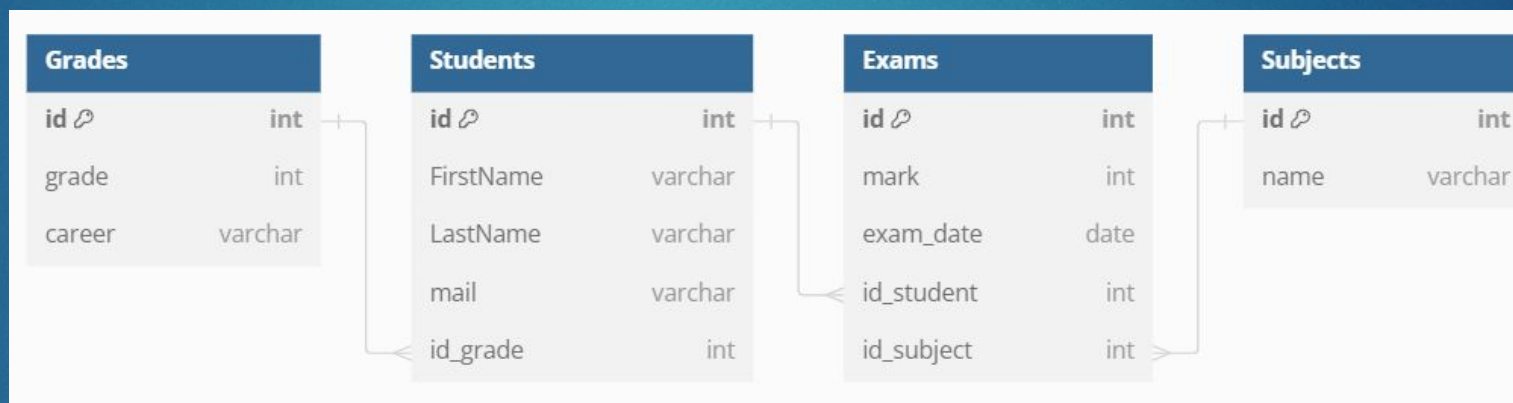
También existen bases de datos no relacionales que no tienen porque estructurar su información en tablas, filas y columnas.



Creación de tablas

Lo primero que necesitamos para comenzar a trabajar con una base de datos es diseñar y crear las varias tablas que necesitaremos para el proyecto.

Para lo cual, uno primero piensa y arma el diagrama junto al equipo de su base de datos. En este diagrama, se deben poder ver los tipos de datos de las filas y las relaciones entre tablas.



Creación de tablas

Para la creación de una tabla, es importante aclarar que tipo de dato será cada columna, cual es la **primary key** y si hay **claves foráneas**.

Si hay claves foráneas, es importante que esas tablas ya existan. Por ejemplo, viendo el diagrama anterior no puedo crear la tabla Students si no existe la tabla Grades.

Entonces creemos la tabla Grades, primero:

```
CREATE TABLE IF NOT EXISTS Grades(  
    id int auto_increment unique NOT null,  
    grade int,  
    career varchar(255),  
    PRIMARY KEY (id)  
);
```

El "if not exists" es para que el comando no se ejecute si la tabla ya existe

Como el id va a ser la clave primaria, es importante que sea UNIQUE para que no se repita y por comodidad, también es muy útil que sea AUTO_INCREMENT

Al final del create, **siempre se aclaran las claves primarias y foráneas**

Creación de tablas

Una vez, creada la tabla Grades, podemos crear la tabla Students que tiene una clave foránea que referencia al id de Grades.

```
CREATE TABLE IF NOT EXISTS Students(  
  id int auto_increment unique NOT null,  
  FirstName varchar(255),  
  LastName varchar(255),  
  mail varchar(255),  
  id_grade int,  
  PRIMARY KEY (id),  
  FOREIGN KEY (id_grade) REFERENCES Grades(id)  
);
```

Los tipos de datos que estoy utilizando son:
int para los números,
varchar(255) para el texto (con un máximo de 255 caracteres)

La clave foránea primero tiene que ser creada como una columna más con el mismo tipo de dato que en la tabla a la que se va a referenciar.

Insertar datos

Luego de crear las tablas, lo que sigue es insertar datos.

Para lo cual se utiliza la instrucción "INSERT INTO". Y se arma de la siguiente manera: INSERT INTO TABLE (COL1, COL2) VALUES(DATA1, DATA2).

Si el dato a agregar es una clave foránea, el valor insertado debe existir en la tabla a la que hace referencia:

```
INSERT INTO Exams (mark, exam_date, id_student, id_subject) VALUES (7, '2022-11-26', 1, 4);
```

También se pueden insertar varios valores en una sola instrucción:

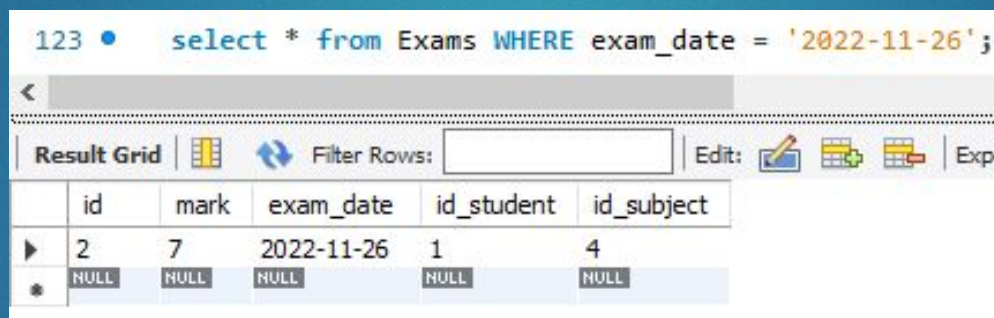
```
INSERT INTO Grades (grade, career) VALUES  
(1, 'Ingenieria Electronica'),  
(2, 'Ingenieria Electronica'),  
(3, 'Ingenieria Electronica'),  
(4, 'Ingenieria Electronica');
```

Actualizar datos

En caso de querer actualizar un dato que ya está en la tabla existe la instrucción “UPDATE” a la cual hay que especificarle la tabla, la columna a cambiar y su nuevo valor.

También es importante agregar una cláusula where que se cumpla únicamente en el dato que queremos modificar.

Por ejemplo, voy a actualizar la nota del examen que pusimos antes:



The screenshot shows a database query interface. At the top, a query editor displays the SQL statement: `select * from Exams WHERE exam_date = '2022-11-26';`. Below the editor, a toolbar includes a 'Result Grid' button, a 'Filter Rows' input field, and an 'Edit' button. The main area displays a table with the following data:

	id	mark	exam_date	id_student	id_subject
▶	2	7	2022-11-26	1	4
*	NULL	NULL	NULL	NULL	NULL

Es una buena práctica usar el id en el WHERE que sabes que no se repite:

```
UPDATE Exams SET mark = 6 WHERE id = 2;
```


Eliminar datos

Por último, si queremos eliminar datos tenemos la sentencia DELETE, a la cual, únicamente hay que pasarle una cláusula WHERE.

```
DELETE FROM Exams WHERE id = 2;
```

Es muy importante a la hora de usar esta instrucción que tengamos cuidado de especificar bien la condición del WHERE, ya que de no usarlo terminaríamos borrando varios datos que no queríamos, y en el peor caso, la tabla completa.

The background of the slide is a dark blue image of a multi-lane highway. Light trails from vehicles are visible, creating a sense of motion. A solid yellow rectangle is positioned in the top right corner.

Gracias